dependencies. When the DI container creates the `LinkGenerator`, it first creates all the service's dependencies and uses them to create the final `LinkGenerator` instance.

Injecting services into your handlers is the canonical DI approach for minimal API endpoint handlers, but sometimes you need to access a service outside the context of a request. You may have lots of reasons to do this, but some of the most common relate to working with a database or logging. You may want to run some code when your app is starting to update a database's schema before the app starts handling requests, for example. If you need to access services in Program.cs outside the context of a request, you can retrieve services from the DI container directly by using the `WebApplication .Services` property, which exposes the container as an `IServiceProvider`.

NOTE You *register* services with the `IServiceCollection` exposed on `WebApplicationBuilder.Services`. You *request* services with the `IServiceProvider` exposed on `WebApplication.Services`.

The `IServiceProvider` acts as a service locator, so you can request services from it directly by using `GetService()` and `GetRequiredService()`:

- `GetService<T>()`—Returns the requested service `T` if it is available in the DI container; otherwise, returns `null`
- `GetRequiredService<T>()`—Returns the requested service `T` if it is available in the DI container; otherwise, throws an `InvalidOperationException`